

Unit V

5

Synchronization and Concurrency Control

5.1 : Concurrency

Q.1 Explain the principle of concurrency.

Ans. : • Concurrent access to shared data may result in data inconsistency. Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.

• Concurrency arises in the same way at different levels of execution streams. Following are the example of concurrency in different types of operating systems :

1. **Concurrenrrency in multiprogramming :** An interaction between multiple processes running on one CPU.
2. **Concucy in multithreading :** An interaction between multiple threads running in one process.
3. **Concurrency in multiprocessors :** An interaction between multiple CPUs running multiple processes or threads.
4. **Concurrency in multi-computers :** An interaction between multiple computers running distributed processes or threads.

- Java is a concurrent programming language.
- Process synchronization is required in uni-processor system, multiprocessor system and network. If more than one thread exists in a system at the same time, then the threads are said to be concurrent. Synchronization problems can occur whenever two or more concurrent processes use any shared resource.

• Cooperating process share the logical address space.

Is concurrency is possible without parallelism ?

- Yes, concurrency is possible without parallelism.

- Concurrency means that more than one process or thread is progressing at the same time. However, it does not imply that the processes are running simultaneously. The scheduling of tasks allows for concurrency, but parallelism is supported only on systems with more than one processing core.

Q.2 Explain the following terms : i) Critical section ii) Race condition

Ans. : i) Critical Section :

- A critical section is a block of code that only one process at a time can execute; so, when one process is in its critical section, no other process may be in its critical section. The critical section problem is to ensure that only one process at a time is allowed to be operating in its critical section.

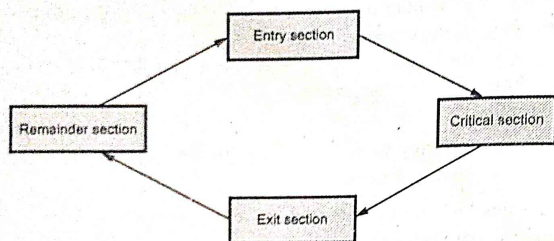


Fig. Q.2.1 Critical section

- Critical section means, process may change some common variable, writing files, updating memory location, updating a process table etc. When process is accessing shared modifiable data, it is said to be in a critical section.
- Each process takes permission from operating system to enter into the critical section. Structure of process is as follows :
 1. Entry section
 2. Remainder section
 3. Exit section.
- **Entry section :** It is block of code executed in preparation for entering critical section.
- **Exit section :** The code executed upon leaving the critical section.
- **Remainder section :** Rest of the code is remainder section.

- Each process cycles through remainder, entry, critical, exit sections in this order.

ii) Race Condition :

- **Race condition** occurs when two or more operations occur in an undefined manner. When two or more processes are reading or writing some shared data and the final result depends on who runs precisely when, are called **race condition**.
- There is a "race condition" if the outcome depends on the order of the execution. The outcome depends on the CPU scheduling or "interleaving" of the threads.
- **Race condition** should be avoided because they can cause fine errors in applications and are difficult to debug.
- In banking system **balance** is a shared variable. After depositing the amount, it update by bank employee
 $\text{Balance} = \text{Balance} + \text{Amount}.$
 At the same time, some amount is transfer then it becomes
 $\text{Balance} = \text{Balance} - \text{Amount}.$
 These two tasks are in a race to write variable **balance**. In this the process that updates last determines the final value of **balance**.
- The concurrent execution of two processes is not guaranteed to be determinate, since different executions of the same program on the same data may not produce the same result.

Q.3 Explain the following terms :

i) Mutual exclusion ii) Synchronization

Ans. : i) Mutual exclusion : • Mutual exclusion methods are used in concurrent programming to avoid the simultaneous use of a common resource, such as a global variable, by pieces of computer code called **critical sections**.

- Requirement of mutual exclusion is that, when process P1 is accessing a shared resource R1 then no other process should be able to access resource R1 until process P1 has finished its operation with resource R1.

- Examples of such resources include files, I/O devices such as printers and shared data structures.

ii) Synchronization : Synchronization means sharing system resources by processes in such way that, concurrent access to shared data is handled thereby minimizing the chance of inconsistent data.

Maintaining data consistency demands mechanisms to ensure synchronized execution of cooperating processes. Process synchronization was introduced to handle problems that arose while multiple process executions.

Q.4 List the requirements of mutual exclusion.

[SPPU : May-16, Dec-17, End Sem, Marks 5]

Ans. : Requirements of mutual exclusion

1. At any time, only one process is allowed to enter in its critical section.
2. Solution is implemented purely in software on a machine.
3. A process remains inside its critical section for a bounded time only.
4. No assumption can be made about relative speeds of asynchronous concurrent processes.
5. A process cannot prevent any other process for entering into critical section.
6. A process must not be indefinitely postponed from entering its critical section.

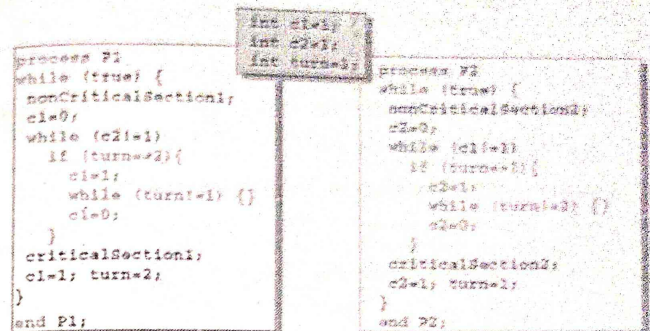
5.2 : Mutual Exclusion : Hardware and Software Approach

Q.5 How livelock situation is avoided by using Dekker's solution ?

Ans. : • Dekker's solution avoids a livelock situation.

- Dekker's algorithm was the first correct solution to the critical section problem for two processes/threads. It uses an array of Boolean values and an integer variable.
- Dekker's algorithm is purely software solution with no special purpose hardware instruction. It uses flag to indicate a process desire to enter its critical section
- If two processes try to enter their critical sections at the same time, variable "turn" indicates which of the processes should be allowed to enter.
- It allows two threads to share a single-use resource without conflict, using only shared memory for communication.
- Processes are loops indefinitely, repeating entering and re-entering its critical section. Here flag and turn are the two global variables.

- Variable flag indicates the position of the process with respect to mutual exclusion. The turn variable is used for resolving the conflicts.
- Algorithm is as follows :



- Here, if P1 finds c2 = 0, it defers to P2 only if turn = 2; otherwise, it simply waits for c2 to become 1 before entering its critical section.
- Process P2, which is also trying to enter its critical section at the same time, is forced to defer to P1 only if turn = 1.
 - Dekker's algorithm is correct and it satisfies mutual exclusion and it is free from deadlock and starvation
 - **Limitation of Dekker's Algorithm :**
 1. Dekker's algorithm does not provide strict alternation.
 2. Dekker's algorithm will not work with many modern CPUs.
 3. This algorithm won't work on Symmetric Multi-Processors (SMP) CPUs.
 4. Additionally, Dekker's algorithm can fail regardless of platform due to many optimizing compilers. Compiler may remove writes to flag since never accessed in loop. Creating an infinite loop.

Q.6 Write short note on : Peterson's solution.

Ans. : • Peterson's solution is software based solution for critical section problem. This algorithm will not work correctly on the modern computer architecture. It is simpler algorithm for two process mutual exclusion with busy waiting.

- This algorithm is less complicated than Dekker's algorithm.
- Peterson's algorithm solves critical section problem of two processes only. It does not require any special hardware. Two processes alternate execution between their critical section and remainder sections.
- Deadlock and indefinite postponement are impossible in Peterson's algorithm as long as no process or thread terminates unexpectedly.
- For indefinite postponement to occur, one process would have to be able to continually complete and reenter its critical section while the other process busy waited.

Q.7 Explain interrupt disabling concept of mutual exclusion.

Ans. : • On uniprocessor system, parallel execution of process is not possible. But they can only be interleaved. To guarantee mutual exclusion, it is sufficient to prevent a process from being interrupted.

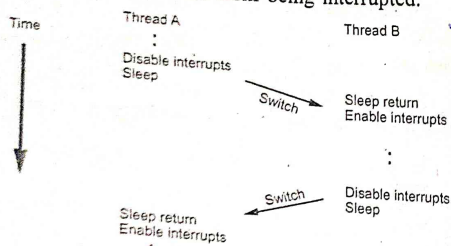


Fig. Q.7.1

- A process can prevent the operating system from executing by not issuing any supervisor call instructions and by disabling interrupts. Access to a critical section can be controlled by calling those primitives.
- Use interrupts,
 - a) Implement preemptive CPU scheduling
 - b) Internal events to relinquish the CPU
 - c) External events to reschedule the CPU.
- Disable interrupts,
 - a) Introduce uninterruptible code regions
 - b) Think sequentially most of the time
 - c) Delay handling of external events.

- Mutual exclusion is guaranteed because the critical section cannot be interrupted

Disadvantages

1. It works only in a single processor environment.
2. Interrupts can be lost if not serviced promptly.
3. A process waiting to enter its critical section could suffer from starvation.

Q.8 What are the advantages and disadvantages of test and set operations ?

Ans. : Advantages :

1. Simple to implement.
2. It works well for a small number of processes.
3. It can be used to support multiple critical sections.

Drawbacks :

1. It suffers from starvation.
2. There is possibility of busy waiting.
3. There may be deadlock.

5.3 : Semaphore

Q.9 Explain with definition, the concept of general and binary semaphore.

Ans. : Concept of general semaphore :

- Semaphore is described by Dijkstra. Semaphore is a nonnegative integer variable that is used as a flag. Semaphore is an operating system abstract data type. It takes only integer value. Its used to solve critical section problem.
- Dijkstra introduced two operations (P and V) to operate on semaphore to solve process synchronization problem. A process calls the P operation when it wants to enter its critical section and calls V operation when it wants to exit its critical section. The P operation is called as wait operation and V operation is called as signal operation.
- A wait operation on a semaphore decrease its value by one.

waits : while $S < 0$

do loops;

$S := S - 1;$

- A signal operation increments its value :

signal :

$S := S + 1;$

- A proper semaphore implementation requires that P and V be indivisible operations. A semaphore operation is atomic. This may be possible by taking hardware support. The operation P and V are executed by the operating system in response to calls issued by any one process naming a semaphore as parameter.
- There is no guarantee that no two processes can execute wait and signal operations on the same semaphore at the same time.

Binary semaphore :

- Binary semaphore is also known as mutex locks. It deals with the critical section for multiple processes. Binary semaphore value is only between 0 and 1.
- 1. The semaphore value is restricted to 0 and 1.
- 2. P succeeds only when the semaphore value is 1.
- 3. V does not change the semaphore value when it is 1.
- Binary semaphores are easier to implement than general semaphores

```
var
    mutex=1:binary-semaphore;
    delay=0:binary-semaphore;
    C={initvalue}:integer;
```

Procedure Wait()

```
begin
    wait(mutex);
    C:=C-1;
    If C < 0 then begin
        signal(mutex);
        wait(delay);
    end
    else
        signal(mu
end
```

Procedure Signal()

```
begin
    wait(mutex);
    C:=C+1;
    If C <= 0 then
        signal(delay)
    end
    signal(mutex)
end
```

Q.10 List the properties of semaphore.

Ans. : Properties of semaphore :

1. Semaphores are machine independent.
2. Semaphores are simple to implement.
3. Correctness is easy to determine.
4. Semaphore acquire many resources simultaneously.
5. Can have many different critical sections with different semaphores.

Q.11 Explain the following functions (along with parameters passed) with reference to semaphore programming in 'C'.

i) sem_post() ii) sem_wait()

Ans. : i) sem_post () :

Syntax :

```
#include <semaphore.h>
```

```
int sem_post(sem_t *sem);
```

- The sem_post() function unlocks the specified semaphore by performing a semaphore unlock operation on that semaphore. When this operation results in a positive semaphore value, no threads were blocked waiting for the semaphore to be unlocked; the semaphore value is simply incremented.
- When the operation results in a semaphore value of zero, one of the threads waiting for the semaphore is allowed to return successfully from its invocation of the sem_wait() function. The sem_post() interface is reentrant with respect to signals and may be invoked from a signal-catching function.

ii) Sem_wait ()

Syntax :

```
int sem_wait(sem_t *s) {
    wait until value of semaphore s is greater
    than 0
    decrement the value of semaphore s by 1
}
```

- The sem_wait () function locks the semaphore referenced by sem by performing a semaphore lock operation on that semaphore.
- If the semaphore value is currently zero, then the calling thread will not return from the call to sem_wait () until it either locks the semaphore or the call is interrupted by a signal.

Q.12 Explain busy waiting with appropriate example ?

[SPPU : Aug-17, In Sem, Marks 4]

OR What is busy waiting with respect to process synchronization ? Explain how semaphore solves problem of synchronization.

[SPPU : Oct-19, In Sem, Marks 6]

Ans. : • Busy waiting is a situation in which a process is blocked on a resource but does not yield the processor. A busy wait keeps the CPU busy in executing a process even as the process does nothing.

- Busy waiting is also called as spin waiting.
- Busy waiting waste CPU cycles that some other process might be able to use productively.
- To avoid busy waiting, a process waiting for critical section, they put into the blocked state. When process allowed entering into critical section, then process is kept in ready state.
- To overcome the need for busy waiting :

In wait () operation :

- When process reads semaphore value and it is not positive value then process blocked itself for some time.
- Semaphore keeps all blocked process in the waiting queue so process is kept in this queue.
- CPU scheduler selects another process for execution.

In signal () operation :

- The wakeup () operation is used for restating the blocked processes. When any other process executes signal () operation, then blocked process state changed from blocked state to ready state.
- This process is kept in the ready queue.

5.4 : Monitor

Q.13 Explain monitor in brief.

[SPPU : May-18,19, Dec.-19, End Sem, Marks 5]

Ans. : • Monitor is an object that contains both data and procedures needed to perform allocation of a shared resource. It is a programming language construct that support both data access synchronization and control synchronization.

- Fig. Q.13.1 shows structure of monitor.

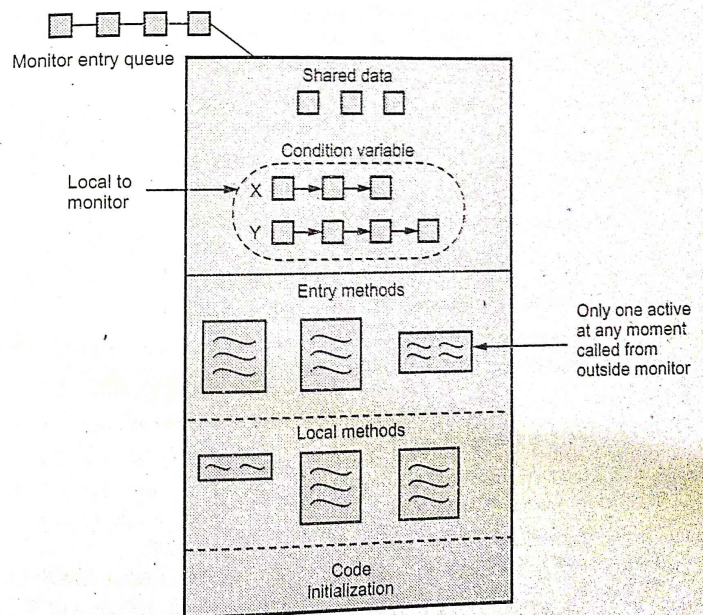


Fig. Q.13.1 Monitor structure

- Monitor is implemented in programming languages like Pascal, Java and C++. Java makes extensive use of monitors to implement mutual exclusion.
- Monitor is an abstract data type for which only one process may be executing a procedure at any given time. Monitor is a collection of procedure, variables and data structure. Data inside the monitor may be either global to all routines within the monitor or local to a specific routine.
- Monitor data is accessible only within the monitor. A shared data structure can be generated by placing it in a monitor. If the data in a monitor represents some resource, then the monitor provides a mutual exclusion facility for accessing the resource. A procedure defined within a monitor can access only those variable declared locally within the monitor and its formal parameters.
- When a process calls a monitor procedure, the first few instruction of the procedure will check to see if any other process is currently active within the monitor. If process is active then calling process will be suspended until the other process has left the monitor. If no other process is using the monitor, the calling process may enter.
- Monitor supports synchronization by the use of condition variables that are contained within the monitor and accessible only within the monitor. Every conditional variable has an associated queue. Condition variables operates on two functions:

cwait(condition variable)

csignal(condition variable)

- *cwait*: It suspend execution of the calling process on condition.
- *csignal*: Resume execution of some process blocked after a *cwait* on the same condition.
- The *cwait* must come before the *csignal*.

Q.14 List the drawback of monitor.

- Ans.:**
1. Major weakness of monitors is the absence of concurrency if a monitor encapsulates the resource, since only one process can be active within a monitor at a time.
 2. There is the possibility of deadlocks in the case of nested monitors calls.

3. Monitor concept is its lack of implementation most commonly used programming languages.
4. Monitors cannot easily be added if they are not natively supported by the language.

Q.15 Compare and contrast the use of monitors and semaphores operations.

Ans.: Difference between Monitors and Semaphores

- A semaphore is an operating system abstract data type whereas monitors are based on abstract data types.
- Semaphore-level synchronization primitives are difficult to use for complex synchronization situations. Monitors were devised to simplify the complexity of synchronization problems by abstracting away details.
- Semaphores provide a general-purpose mechanism for controlling access to critical sections. Their use does not guarantee that access will be mutually exclusive or deadlock will be avoided. A monitor is a programming language construct that guarantees appropriate access to critical sections.
- Monitor uses condition variables. A condition variable is like a semaphore, with two differences,
 - 1) A semaphore counts the number of excess up operations, but a signal operation on a condition variable has no effect unless some process is waiting. A wait on a condition variable always blocks the calling process.
 - 2) A wait on a condition variable automatically does an up on the monitor and blocks the caller.

5.5 : Reader Writer Problem

Q.16 Explain reader writer problem.

Ans.: • When two types of processes need to access a shared resource such as a file or database. They called these processes reader and writers.

- Multiple readers can concurrently read from the data base. But when updating the database, there can only be one writer (i.e., no other writers and no readers either)
- More than one reader may read shared data (no writers). When a writer uses shared data, all other writers and readers must be excluded.

- Fig. Q.16.1 shows reader-writer problem.

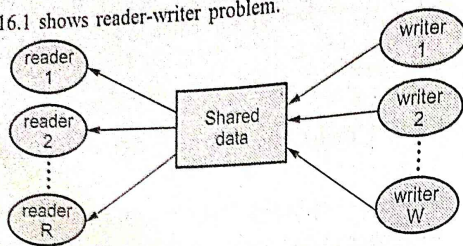


Fig. Q.16.1 Reader-writer problem

- For example in railway reservation system, readers are those who want train schedule information. They are called reader because readers do not change the content of the database. Many readers may access the database at once. There is no need to enforce mutual exclusion among readers.
- The writers are those who making reservations on a particular train. Writer can modify the database, so it must have exclusive access.
- When writer is active, no other readers or writers may be active. Therefore, it must enforce mutual exclusion if there are groups of readers and a writer.
- Reader writer problem is similar to one which a file is to be shared among a set of processes. If a process want only to read the file, then it may share the file with any other process that also wants to read the file.
- We apply rules for access order :
 - If a writer is waiting for readers to be finished, do not allow any more readers.
 - If a reader is waiting for a writer to finish, give reader priority.
- If the writer wants to append the file, then no other process should have access to the file when the writer has access to it.
- If reader having higher priority than the writer, then there will be starvation with writers. For writer having higher priority than reader then starvation with readers.

Q.17 Write a semaphore solution for readers - writer's problem.

[SPPU : May-18, Dec.-18, End Sem, Marks 5]

Ans. : • When two types of processes need to access a shared resource such as a file or database. They called these processes reader and writers.

- The writers are those who making reservations on a particular train. Writer can modify the database, so it must have exclusive access. When writer is active, no other readers or writers may be active. Therefore, it must enforce mutual exclusion if there are groups of readers and a writer.
- Reader writer problem is similar to one which a file is to be shared among a set of processes. If a process want only to read the file, then it may share the file with any other process that also wants to read the file. If the writer wants to append the file, then no other process should have access to the file when the writer has access to it.

```
int readcount = 0;
semaphore wsem = 1;
semaphore x = 1;
void main( )
{
    int p = fork( );
    if(p reader;           // assume multiple instances
    else writer;           // assume multiple instances
}

void reader( )
{
    while(1)
    {
        wait(x);
        readcount ++;
        if (readcount == 1)
            wait (wsem);
        signal(x);
        doReading( );
        wait(x);
        readcount--;
        if (readcount==0)
            signal(wsem);
        signal(x);
    }
}

void writer( ) {
    while(1) {
        wait(wsem)
        doWriting( );
        signal(wsem)
    }
}
```


5.6 : Producer Consumer Problem

Q.18 Write a pseudo code for producer-consumer problem using semaphore.

[SPPU : Aug-17, In Sem, Marks 6]

Ans. : • Solution to producer consumer problem by using binary semaphore :

```
void main ()
{
    count = 0;
}
int i;
binarysemaphore s = 1;
int producer ()
{
    While (true)
    {
        produce_data_item ();
        semwaitB(s);
        append ();
        count = count + 1;
        if ( count == 1)
            semSignalB(delay);
        semSignalB(s);
    }
}
int consumer ()
{
    int p;
    semWaitB(delay);
    while (true)
    {
        semWaitB(s);
        consume();
        count = count - 1;
        p = count;
        semSignalB(s);
        consumedata();
        if ( p == 0)
            semWaitB(delay);
    }
}
```

Q.19 What is critical section ? Give semaphore solution for producer - consumer problem.

Ans. : • A critical section is a block of code that only one process at a time can execute; so, when one process is in its critical section, no other process may be in its critical section. The critical section problem is to ensure that only one process at a time is allowed to be operating in its critical section.

- Producer - consumer problem is example classic problems of synchronization. Producer process produce data item that consumer process consumes later.
- Buffer is used between producer and consumer. Buffer size may be fixed or variable. The producer portion of the application generates data and stores it in a buffer and the consumer reads data from the buffer.
- In order to synchronize these processes, both procedure and consumer are blocked on some condition. The producer is blocked when the buffer is full and the consumer is blocked when the buffer is empty.

1. Code for producer	2. Code for consumer process
<pre>producer (void) { int item; while (TRUE) { produce item (& item); produce_if (counter==N) sleep (); enter_item (item); counter = counter + 1; if (counter == 1) wakeup (consumer); } }</pre>	<pre>consumer (void) { int item; while (TRUE) { if (count == 0) sleep (); remove_item (& item); counter = counter - 1; if (count == N - 1) wakeup (producer); consume_item (item); } }</pre>

5.7 : Dining Philosopher Problem

Q.20 Write semaphore solution for dining Philosophers problem.

[SPPU : Dec.-17,19, End Sem, Marks 5]

Ans. : • There is one semaphore for each chopstick. In addition, a local two-phase prioritization scheme is used, under which philosophers defer to their neighbors who have declared themselves "hungry." All arithmetic is modulo 5.

system DINING_PHILOSOPHERS

VAR

```
me: semaphore, initially 1; /* for mutual exclusion */
s[5]: semaphore s[5], initially 0; /* for synchronization */
pflag[5]: {THINK, HUNGRY, EAT}, initially THINK;
/* philosopher flag */
```

// As before, each philosopher is an endless cycle of thinking and eating.

procedure philosopher(i)

```
{
  while TRUE do
  {
    THINKING;
    take_chopsticks(i);
    EATING;
    drop_chopsticks(i);
  }
}
```

// The take_chopsticks procedure involves checking the status of neighboring philosophers and then declaring one's own intention to eat. This is a two-phase protocol; first declaring the status HUNGRY, then going on to EAT.

procedure take_chopsticks(i)

```
{
  DOWN(me); /* critical section */
  pflag[i] := HUNGRY;
```

DECODE

```
test[i];
UP(me); /* end critical section */
DOWN(s[i]) /* Eat if enabled */
}
void test(i) /* Let phil[i] eat, if waiting */
{
  if ( pflag[i] == HUNGRY
    && pflag[i-1] != EAT
    && pflag[i+1] != EAT)
  then
  {
    pflag[i] := EAT;
    UP(s[i])
  }
}
```

// Once a philosopher finishes eating, all that remains is to relinquish the resources, its two chopsticks and thereby release waiting neighbors.

void drop_chopsticks(int i)

```
{
  DOWN(me); /* critical section */
  test(i-1); /* Let phil. on left eat if possible */
  test(i+1); /* Let phil. on right eat if possible */
  UP(me); /* up critical section */
}
```

Q.21 Explain classical problem of synchronization in terms of Dining Philosopher problem. [SPPU : May-17, End Sem, Marks 5]

Ans. : • Dining philosophers problem is one of classical process synchronization problem. Here five philosophers are seated around a circular table. They spend their lives in thinking and eating. Five plates with five forks are kept on the circular table.

• While philosophers think, they ignore the eating and do not require a fork. When a philosopher decides to eat, then he/she must obtain a two fork, one from left side fork and another from right side fork. Philosophers can pick up only a single fork at one time.

DECODE

- After consuming a food, the philosophers replace the fork and resumes thinking. A philosopher to the left or right of a dining philosopher cannot eat while the dining philosopher is eating, since forks are a shared resource.
- Fig. Q.21.1 shows seating arrangement of five philosophers.

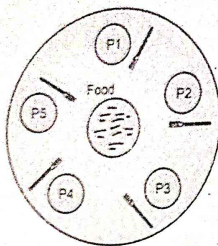


Fig. Q.21.1 Dining philosophers

- Consider the following code :

```

void dinning_Philosopher ()
{
    while (true)
    {
        thinking ( );
        eating ( );
    }
}

void eating ( )
{
    takeleftfork ( );
    takerightfork ( );
    eatfood(delay);
    putrightfork ( );
    putleftfork ( );
}

```

- Here philosophers operates asynchronously and concurrently, it is possible for each philosophers to enter into eating mode. The procedure takeleftfork () waits until the specified fork is available and then seizes it. This is not possible because of the following reasons.

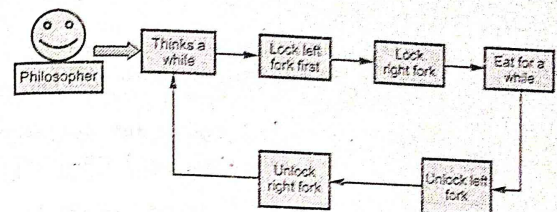


Fig. Q.21.2 Philosopher states

- Each philosopher will hold exactly one fork, and no forks will remain available on the table. The philosophers will all deadlock.
- To solve this problem, write some extra code in takerightfork () procedure. User can specify the philosophers to put down the left fork if that philosophers cannot obtain the right fork.

Problem analysis :

- **Shared resource :** Here each fork is shared by two philosophers so we called it is a shared resource.
- **Race condition :** We do not want a philosopher to pick up a fork that has already been picked up by his neighbor. This is a race condition.
- **Solution :** We consider each fork as a shared item and protected by a mutex lock. Before eating, each philosopher first lock left fork and then right fork. If philosopher acquires both locks successful, then philosophers have two locks (two fork) so he can eat a food. After finishes eating, this philosopher releases both fork, and thinks.

Some other alternatives :

1. Pick up the left fork, if the right fork is not available for a given time, put the left fork down, wait and try again. Even if each philosopher waits a different random time, an unlucky philosopher may starve.
2. Require all philosophers to acquire a binary semaphore before picking up any forks. This guarantees that no philosopher starves but limits parallelism dramatically.

- Because we need to lock and unlock a chopstick, each chopstick is associated with a mutex lock. Since we have five philosophers who think and eat simultaneously, we need to create five threads, one for each philosopher. Since each philosopher must have access to the two mutex locks that are associated with its left and right chopsticks, these mutex locks are global variables.

5.8 : Deadlocks : Principle of Deadlock

Q.22 What is deadlock ? State and explain the conditions for deadlock.

[SPPU : May-18, End Sem, Marks 8]

OR What are necessary conditions for deadlock ? Explain them with example.

[SPPU : Dec.-18, End Sem, Marks 4]

Ans. : Deadlock can be defined as the **permanent blocking** of a set of processes that either complete for system resources. Deadlock can occur on sharable resources.

- A process is in deadlock state if it was waiting for a particular event that will not occur. In a system deadlock, one or more processes are deadlocked.

- Following four conditions are necessary for deadlock to exist.

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

1. **Mutual exclusion :** A resource may be acquired exclusively by only one process at a time.

2. **Hold and wait :** Processes currently holding resources that were granted earlier can request new resources.

3. **No preemption :** Once a process has obtained a resource, the system cannot remove it from the process control until the process has finished using the resource.

4. **Circular wait :** A circular chain of hold and wait condition exists in the system.

- All four of these conditions must be present for a resource deadlock to occur. If one of them is absent, no resource deadlock is possible.

Q.23 Differentiate between the following : Deadlock and starvation.

Ans. : • **Deadlocks :** A group of processes are waiting for resources held by others in the group. None of them will ever make progress.

• **Starvation :** A process may wait indefinitely because other process keep coming in and getting the requested resources before this process does.

- In a deadlock situation, none of the involved processes can possibly make progress. In a starvation situation, a process is ready to execute, but it is not being allowed to execute.

Q.24 Write short notes on : Resource allocation graph and resource allocation graph algorithm.

Ans. : Resource allocation graph :

- Resource allocation graph is introduced by Holt. It is a directed graph that depicts a state of the system of resources and processes.



Fig. Q.24.1 Process

- Process and resource are represented by node in directed graph. Graph consists of a set of vertices (V) and set of edges (E).

- A process node is graphically represented by circle.

- A resource node is graphically represented by a rectangle and represents one type of resources.

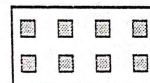


Fig. Q.24.2 Resource with 8 instances

- The number of bullet symbols in a resource node indicates how many units of that resource class exist in the system.
- An arrow from the process to resource indicates the process is requesting the resource. An arrow from resource to process shows an instance of the resource has been allocated to the process.
- Claim edge $P \rightarrow R$ indicated that process P may request resource R in the future ; represented by a dashed line. Claim edge converts to request edge when a process requests a resource.
- Request edge converted to an assignment edge when the resource is allocated to the process. When a resource is released by a process, assignment edge reconverts to a claim edge.



Fig. Q.24.3 Edges

- Fig. Q.24.4 shows a resource allocation graph. System consists of process and resources.

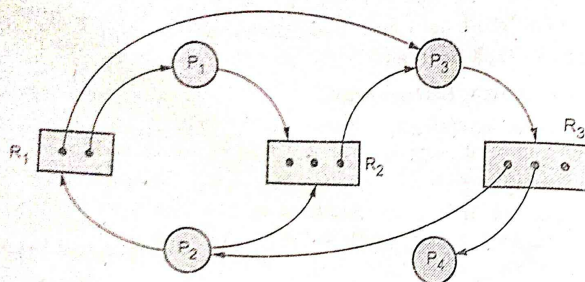


Fig. Q.24.4 Resource allocation graph

Process : P_1, P_2 Resource : R_1, R_2

Resource allocation graph algorithm : In resource-allocation graph, normally request edge and assignment edge is used.

In addition to the request and assignment edge, new edge called **claim edge** is added. For example, a claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future. This claim is shown by dotted lines in the figure. Fig. Q.24.5 shown this deadlock avoidance with resource allocation graph.

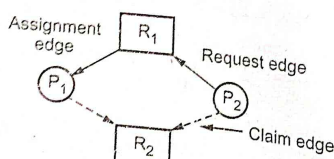


Fig. Q.24.5

- When process P_i request resources R_j , the claim edge is converted to a request edge.
- When a resource R_j is released by P_i , the assignment edge $R_j \rightarrow P_i$ is reconverted to a claim edge $P_i \rightarrow R_j$.
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph.

5.9 : Deadlock Prevention

Q.25 Explain deadlock prevention techniques with example.

[SPPU : May-16, End Sem, Marks 5]

Ans. : • To prevent a deadlock, the OS must eliminate one of the four necessary conditions.

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

1. Mutual exclusion : It is necessary in any computer system because some resources (memory, CPU) must be exclusively allocated to one user at a time. No other process can use a resource while it is allocated to a process.

2. Hold and wait : If a process holding certain resources is denied a further request, it must release its original resources and if required request them again.

3. No preemption : It could be bypassed by allowing the operating system to deallocate resources from process.

4. Circular wait : Circular wait can be bypassed if the operating system prevents the formation of a circle.

• A deadlock is possible only if all four of these conditions simultaneously hold in the system.

• Prevention strategies ensure that at least one of the conditions is always false.

Q.26 How can the hold-and wait condition be prevented ?

Ans. : To prevent hold-and wait condition, processes must be prevented from holding one or more resources while simultaneously waiting for one or more others.

- There are several possibilities for this :

1. Require that all processes request all resources at one time. This can be wasteful of system resources if a process needs one resource early in its execution and doesn't need some other resource much later.
 2. Require that processes holding resources must release them before requesting new resources, and then re-acquire the released resource along with the new ones in a single new request. This can be a problem if a process has partially completed an operation using a resource and then fails to get it re-allocated after releasing it.
- Either of the methods described above can lead to starvation if a process requires one or more popular resources.

5.10 : Deadlock Avoidance

Q.27 Explain deadlock avoidance algorithm with suitable example.

[SPPU : Dec.-19, End Sem, Marks 7]

OR Write Banker's algorithm and explain with suitable example.

[SPPU : Dec.-18, End Sem, Marks 6]

OR Write and explain Banker's algorithm for avoidance of deadlock.

Ans. : • Banker's algorithm is the deadlock avoidance algorithm. Banker's is named because the algorithm is modeled after a banker who makes loans from a pool of capital and receives payments that are returned to that pool.

- Algorithm is check to see if granting the request leads to an unsafe state. If it does, the request is denied. If granting the request leads to a safe state, it is carried out.
- The Dijkstra proposed an algorithm to regulate resource allocation to avoid deadlocks. The banker's algorithm is the best known of the avoidance method.
- By using avoidance method, the system is always kept in a safe state.
- It is easy to check if a deadlock is created by granting a request, deadlock analysis method is used for this.
- Deadlock avoidance uses the worst-case analysis method to check for future deadlocks.

- **Safe state** : There is at least one sequence of resource allocations to processes that does not result in a deadlock.
- System is in a safe state only if there exist a safe sequence. A safe state is not a deadlocked state.
- Deadlocked state is an unsafe state. It does mean the system is in a deadlock.
- As long as the state is safe, the resource manager can be guaranteed to avoid a deadlock.
- Initially the system is in a safe state. When process requests a resource and that resource is available then the system must decide whether the resources can be allocated immediately or process must wait.
- If system remains in safe state after allocating resource then only OS allocates resources to process.
- Banker algorithm uses following data structures.

1. **Allocation** : Allocation is a table in which row represents process and column represents resources (R).

$\text{alloc}[i, j] = \text{Number of unit of resource } R_j \text{ held by process } P_i.$

2. **Max** : Max be the maximum number of resources that process requires during its execution.

- **Need (claim)** : It is current claim of a process, where a process's claim is equal to its maximum need minus its current allocation.

$$\text{Need} = \text{Max} - \text{Allocation}$$

- **Available** : There will be number of resources still available for allocation. This is equivalent to the total number of resources minus the sum of the allocation to all processes in the system.

$\text{Available} = \text{Number of resources} - \text{Sum of the allocation to all process}$

$$= \text{Number of resources} - \sum_{i=1}^n \text{Allocation}(P_i)$$

- Each process cannot request more that the total number of resources in the system. Each process must also guarantee that once allocated a resource, the process will return that resource to the system within a finite time.

Weakness of Banker's algorithm

1. It requires that there be a fixed number of resources to allocate.
2. The algorithm requires that users state their maximum needs (request) in advance.
3. Number of users must remain fixed.
4. The algorithm requires that the bankers grant all requests within a finite time.
5. Algorithm requires that process returns all resource within a finite time.

Q.28 Consider the following snapshot of a system.

Processes	Allocation				Max.				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P ₀	2	0	0	1	4	2	1	2	3	3	2	1
P ₁	3	1	2	1	5	2	5	2				
P ₂	2	1	0	3	2	3	1	6				
P ₃	1	3	1	2	1	4	2	4				
P ₄	1	4	3	2	3	6	6	5				

Answer the following question using Banker's algorithm.

- Illustrate that the system is in safe state by demonstrating an order in which the processes may complete?
- If a request from a process P₁ arrives for (1, 1, 0, 0), can the request be granted immediately?
- If the request from P₄ arrives for (0, 0, 2, 0), can the request be granted immediately?

Ans. : Need Matrix :

	A	B	C	D
P ₀	2	2	1	1
P ₁	2	1	3	1
P ₂	0	2	1	3
P ₃	0	1	1	2
P ₄	2	2	3	3

i) Safe State :

Process	Need	Available	T/F	Available
P ₀	(2, 2, 1, 1)	< (3, 3, 2, 1)	T	(2, 0, 0, 1) + (3, 3, 2, 1) = (5, 3, 2, 2)
P ₁	(2, 1, 3, 1)	> (5, 3, 2, 2)	F	(5, 3, 2, 2)
P ₂	(0, 2, 1, 3)	> (5, 3, 2, 2)	F	(5, 3, 2, 2)
P ₃	(0, 1, 1, 2)	< (5, 3, 2, 2)	T	(1, 3, 1, 2) + (5, 3, 2, 2) = (6, 6, 3, 4)
P ₄	(2, 2, 3, 3)	< (6, 6, 3, 4)	T	(1, 4, 3, 2) + (6, 6, 3, 4) = (7, 10, 6, 6)

Second iteration :

P ₁	(2, 1, 3, 1)	< (7, 10, 6, 6)	T	(3, 1, 2, 1) + (7, 10, 6, 6) = (10, 11, 8, 7)
P ₂	(0, 2, 1, 3)	> (10, 11, 8, 7)	T	(2, 1, 0, 3) + (10, 11, 8, 7) = (12, 12, 8, 10)

Safe sequence : P₀, P₃, P₄, P₁, P₂

ii) Request from process P₁ (1, 1, 0, 0)

We assume that, requested is granted immediately. So calculate need matrix :

	A	B	C	D
P ₀	2	2	1	1
P ₁	2 - 1 = 1	1 - 1 = 0	3 - 0 = 3	1 - 0 = 1
P ₂	0	2	1	3
P ₃	0	1	1	2
P ₄	2	2	3	3

Available = (2, 2, 2, 1)

Calculate safe sequence :

Process	Need	Available	T/F	Available
P ₀	(2, 2, 1, 1)	= (2, 2, 2, 1)		unsafe state
P ₁	(1, 0, 3, 1)	> (2, 2, 2, 1)	F	(2, 2, 2, 1)
P ₂	(0, 2, 1, 3)	> (2, 2, 2, 1)	F	(2, 2, 2, 1)
P ₃	(0, 1, 1, 2)	< (2, 2, 2, 1)	F	(2, 2, 2, 1)
P ₄	(2, 2, 3, 3)	< (2, 2, 2, 1)	F	(2, 2, 2, 1)

There is no safe sequence, so request is not granted immediately.

iii) Request from process P₄ (0, 0, 2, 0)

We assume that, request is granted immediately. So calculate need matrix :

	A	B	C	D
P ₀	2	2	1	1
P ₁	2	1	3	1
P ₂	0	2	1	3
P ₃	0	1	1	2
P ₄	2 - 0 = 2	2 - 0 = 2	3 - 2 = 1	3 - 0 = 3

Available = (3, 3, 0, 1)

There is no safe sequence, so request is not granted immediately.

Q.29 Consider following status of system, compute need of each process and check given state is safe or unsafe using Bankers algorithm. Also check, if P₃ generates new request (0, 1, 0, 0) system is safe or unsafe and request is granted or not ?

[SPPU : Dec.-18, End Sem, Marks 8]

	MAX_CLAIM				ALLOCATED			
	R1	R2	R3	R4	R1	R2	R3	R4
P1	0	0	1	2	0	0	1	2
P2	2	7	5	0	2	0	0	0
P3	6	6	5	6	0	0	3	4
P4	4	3	5	6	2	3	5	4
P5	0	6	5	2	0	3	3	2

Currently Available Resources →

R1	R2	R3	R4
2	1	0	0

Ans. : Content of need matrix :

Processes	R1	R2	R3	R4
P1	0	0	0	0
P2	0	7	5	0
P3	6	6	2	2
P4	2	0	1	1
P5	0	3	2	0

System is safe or not :

Available = (2 1 0 0)

Process	Need	Available	Condition	New Available
P1	(0 0 0 0)	(2 1 0 0)	Need < Available	(2 1 1 2)
P2	(0 7 5 0)	(2 1 1 2)	Need > Available	(2 1 1 2)
P3	(6 6 2 2)	(2 1 1 2)	Need > Available	(2 1 1 2)
P4	(2 0 1 1)	(2 1 1 2)	Need < Available	(4 4 6 6)
P5	(0 3 2 0)	(4 4 6 6)	Need < Available	(4 7 9 8)

Again process for process P2 and P3

P2	(0 7 5 0)	(4 7 9 8)	Need < Available	(6 7 9 8)
P3	(6 6 2 2)	(6 7 9 8)	Need < Available	(6 7 12 12)

Safe sequence = P1, P4, P5, P2, P3

- If P3 generates new request (0, 1, 0, 0)

We assume that, request is granted. Then new need matrix will be,

Processes	R1	R2	R3	R4
P1	0	0	0	0
P2	0	7	5	0
P3	6	5	2	2
P4	2	0	1	1
P5	0	3	2	0

Available = (2 0 0 0)

Again calculate safe sequence :

Process	Need	Available	Condition	New Available
P1	(0 0 0 0)	(2 0 0 0)	Need < Available	(2 0 1 2)
P2	(0 7 5 0)	(2 0 1 2)	Need > Available	(2 0 1 2)
P3	(6 5 2 2)	(2 0 1 2)	Need > Available	(2 0 1 2)
P4	(2 0 1 1)	(2 0 1 2)	Need < Available	(4 3 6 6)
P5	(0 3 2 0)	(4 3 6 6)	Need < Available	(4 6 9 8)

Again process for process P2 and P3

P2	(0 7 5 0)	(4 6 9 8)	Need > Available	(4 6 9 8)
P3	(6 6 2 2)	(6 7 9 8)	Need > Available	(4 6 9 8)

So there is no Safe sequence, request is not granted.

5.11 : Deadlock Detection

Q.30 What is deadlock detection?

Ans. : • Deadlock detection is the process of determining that a deadlock exists and identifying the processes and resources involved in the deadlock.

- If processes are blocked on resources for an inordinately long time, the detection algorithm is executed to determine whether the current state is a deadlock.
- Deadlock detection algorithms can incur significant runtime overhead.

Q.31 What is wait for graph ?

Ans. : • Any resource allocation graph with a single copy of resources can be transferred to a wait for graph.

- Fig. Q.31.1 shows resource allocation graph with corresponding wait for graph. The state of the system can be modeled by directed graph, called a wait for graph.

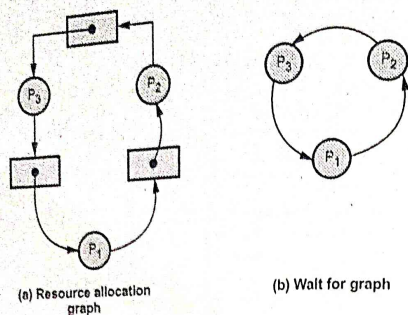


Fig. Q.31.1

- Wait for graph is a graph where each node represents a process. An edge $P_i \rightarrow P_j$ means that process P_i is blocked and waiting for process P_j to release a resource.
- The wait for graph of a system is always smaller than the resource allocation graph of that same system. There is a deadlock in a system if and only if there is a loop in the wait for graph of that system.
- Deadlock detection involves two issues :
 1. Maintenance of the wait for graph.
 2. Searching of the wait for graph for the presence of cycles.
- Deadlock detection requires overhead for run time cost of maintaining necessary information and executing the detection algorithm.
- For multiple instances of a resource type use an algorithm similar to banker's algorithm.

5.12 : Deadlock Recovery

Q.32 What is deadlock ? Explain deadlock recovery methods ?

[SPPU : May-19, End Sem, Marks 6]

Ans. : • Once deadlock has been detected in the system, the deadlock must be broken by removing one or more of the four necessary conditions.

- Here one or more processes will have to be preempted, thus releasing their resources so that the other deadlocked processes can become unblocked.

1. Process termination

- Deadlock is removed by aborting a process. But aborting process is not easy. All deadlocked processes are aborted.
- Circular wait is eliminated by aborting one by one process. There will be lot of overhead. Deadlock detection algorithm must rerun after each process kill.
- Selection of process for aborting is difficult. Following parameters are considered :

1. Priority of the process.
2. What percentage the process finished its execution ?
3. Resource used by process.
4. Need of resources to complete process remaining operation.
5. How many processes will need to be terminated ?
6. Process type : Batch or interactive.

2. Resource preemption

- Some times, resource temporarily take away from its current process and allocate it to another process.
- For selecting victim, following factors are considered,
 1. Priority of the process. Higher priority process are usually not selected.
 2. CPU time used by process. The process which is close to completion are usually not selected.
 3. The number of other process that would be affected if this process were selected as the victim.

3. Recovery through rollback

- When a process in a system terminates, the system performs a rollback by undoing every operation related to the terminated process.
- Checkpointing a process means that its state is written to a file so that it can be restarted later.

- Risk in this method is that the original deadlock may recover but the nondeterminancy of concurrent processing may ensure that this does not happen.

4. Starvation

- Starvation is one type situation in which a process waits for an event that might never occur in the system.
- Select the victim only for finite number of times. Use rollback method for selecting victim process.

END... ✍